SATYA LINUX NOTES

1. Linux Structure

1.1 What is Linux Structure?

Linux follows a **layered architecture**, meaning the system is divided into different levels, each performing a specific function.

Basic Idea:

- Hardware is at the bottom
- Kernel sits above hardware
- User applications sit at the top

This design ensures **abstraction, modularity, and portability**

1.2 Layers of Linux

1. Hardware Layer

- Physical components:
  - CPU
  - Memory (RAM)
  - Disk drives
  - I/O devices

Provides raw computing power

2. Kernel (Core of Linux)

- The **most important part** of Linux
- Directly interacts with hardware

Functions of Kernel:

- Process management
- Memory management
- File system management
- Device control
- Security & permissions

The kernel ensures **efficient resource utilization**

3. System Libraries

- Provide predefined functions
- Help programs interact with kernel

Example: C standard libraries

4. Shell

- Interface between user and kernel
- Accepts commands and executes them

Types of Shell:

- Bash (most common)
- sh
- zsh

Shell converts user commands into **system calls**

5. User Applications

- Programs used by users:
    - Text editors
    - Browsers
    - Compilers

These do not interact with hardware directly

1.3 Working of Linux Structure

Flow:

1. User enters command
2. Shell interprets it
3. System calls sent to kernel
4. Kernel interacts with hardware
5. Output returned to user

This ensures **safe and controlled execution**

1.4 System Calls

System calls are the **bridge between user programs and kernel**.

Examples:

- File operations:
  - open(), read(), write(), close()
- Process control:
  - fork(), exec(), kill()
- Directory operations:
  - mkdir(), chdir()

Programs never access hardware directly—they use system calls

1.5 Key Features of Linux Structure

✓ Multi-user System

- Multiple users can use the system simultaneously

✓ Multi-tasking

- Multiple processes run at the same time

✓ Portability

- Works on different hardware platforms

✓ Security

- Controlled access using permissions

✓ Stability

- Handles high workloads efficiently

1.6 Why Layered Structure is Important

- **Abstraction** → Users don't deal with hardware
- **Security** → Restricted access
- **Maintainability** → Easy to update components
- **Flexibility** → Supports multiple applications

1.7 Simple Diagram (for Exam)

User Applications
↓
Shell
↓
System Libraries
↓
Kernel
↓
Hardware

Draw this in exam → guaranteed marks

1.8 Real-Time Example

When you run:

ls

What happens:

1. User types ls
2. Shell interprets command
3. Kernel processes request
4. Kernel reads directory from disk
5. Output displayed

1.9 Important Exam Points

- Kernel = **heart of Linux**
- System calls = **interface to kernel**
- Linux = **multi-user + multi-tasking**
- Layered design = **modular + secure**